Compatibility and Coupling

Chapter 8 — Software Architecture (Pautasso)

How dependencies, change, and contracts interact

Learning objectives

- Recognize **coupling** and its costs
- **Syntactic** vs **semantic** compatibility
- **Breaking** vs **non-breaking** evolution
- Manage **dependencies** and **version coexistence**

Recap — Lecture 7

Composability and Connectors

- Systems = **components + connectors**
- Sync vs async; **orchestration** vs **choreography**
- → Today: what happens when interfaces and deployments **change**

Recap: connectors & change

- Connectors encode **protocol**, **direction**, **temporal** coupling
- Wrong connector → harder to **evolve** one side without breaking the other
- Events loosen **temporal** coupling but add **schema** and **ordering** concerns

Recap: composition styles

	Orchestration	Choreography
Flow	Central	Distributed via events
Trace	Easier	Harder

Compatibility matters for **every** hand-off between components and clients

Recap: event-driven caveat

- **Schema evolution** for events = public contract
- Producers and consumers **co-evolve** or use **tolerant** readers
- Lecture 8 vocabulary helps you **name** those risks (**coupling**, **compatibility**)

Recap → Chapter 8

- Lecture 7: **how** parts connect
- Lecture 8: **how tightly** they bind, and **whether** a change is **safe** for partners
- Builds on Lecture **6** (interfaces, versioning)

What is coupling?

Coupling = degree to which one part **depends on** another

- Strong coupling → small change **ripples** widely
- Goal is not “zero coupling” but **controlled**, **explicit** dependencies

Why coupling matters

- Affects **cost of change**, **testability**, **deployment independence**
- Architectural decisions (shared DB, shared DTOs, sync chains) **create** coupling
- **Document** dependency direction in diagrams

Kinds of coupling (overview)

Kind	Idea
Data	Shared structures, schemas
Control	Who drives order / calls whom
Temporal	Need to be “ready” at same time
Deployment	Must release together

(Use your chapter’s full taxonomy where it differs.)

Reducing coupling

- **Interfaces** and **boundaries** (ports/adapters)
- **Events** or **messages** vs direct chains (trade-offs)
- **Versioned** APIs; **tolerant** readers; **feature flags**
- Avoid **leaking** internal models across boundaries

Compatibility

Compatible = one party can change **without** forcing immediate coordinated change on others

- **Backward:** new server, old client still works
- **Forward:** old server, new client (rarer; often limited)

Syntactic vs semantic

- **Syntactic** — fields, types, wire format parse
- **Semantic** — meaning, ordering, error interpretation

Same JSON shape can **break** semantics (e.g. stricter validation, different idempotency)

Breaking vs non-breaking (API)

Often **non-breaking**:

- Add **optional** field or parameter with safe default
- Add **new** endpoint

Often **breaking**:

- Remove/rename required fields
- Tighten validation for existing inputs

Semantic versioning (reminder)

- **MAJOR** — incompatible API for consumers
- **MINOR** — backward compatible additions
- **PATCH** — compatible fixes

Align **policy** with what your clients actually tolerate

Version coexistence

- **/v1** and **/v2** paths, **headers**, or gateways
- **Sunset** dates + **deprecation** notices
- Operational cost: dual stacks, routing, docs

Course examples

- `example1_compatibility_and_api_evolution.py`
- `example2_coupling_and_dependencies.py`

Assignment (overview)

Task Board API — coupling analysis, compatibility classification, coexistence, policy

- Full spec: `ASSIGNMENT.md` (deck **does not** solve it)

Assignment: scenario

Multi-client **REST** task API: web, mobile, partners.

You analyze **where coupling hurts**, classify **proposed changes**, plan **v1/v2**, write a **short policy**.

Design / docs only — no full implementation required.

What you produce

Area	Deliverable
Coupling	Analysis + dependency diagram
Compatibility	Per-change: breaking? semver? semantic risk
Coexistence	Strategy for running v1 + v2
Policy	Governance + deprecation
Diagram	Migration / dual-version sequence

Part 1 — Coupling

Task 1.1 — ≥5 elements; dependencies; coupling types; ripple; tight vs reduced

`part1_coupling_analysis.md` — **30 pts**

Task 1.2 — draw.io diagram + legend + PNG — **20 pts**

Part 2 — Compatibility

Task 2.1 — Classify changes A–E (breaking, semver, semantic note)

`part2_compatibility_changes.md` — **25 pts**

Task 2.2 — v1/v2 coexistence strategy + operational cost

`part2_version_coexistence.md` — **15 pts**

Part 3 — Policy & migration

Task 3.1 — Compatibility & deprecation policy (≤2 pages)

`part3_compatibility_policy.md` — **20 pts**

Task 3.2 — Sequence diagram (≥5 participants) + PNG — **10 pts**

Submission & grading

- **PR** to `lecture-8/submissions/YOUR_NAME/` — see `SUBMISSION_GUIDE.md`
- **Total: 120 points**
- Every diagram: **drawio + PNG**

Before you start

1. Run both **Python examples**
2. Draw clients → API → store; mark **who depends on which JSON shape**
3. For each proposed API change, ask: **Would an old client break without code change?**

Takeaways

1. **Coupling** shapes cost of change
2. **Syntactic** safety ≠ **semantic** safety
3. **Versioning** + **coexistence** are architectural choices
4. **Policy** turns ad hoc decisions into team rules

Questions?

Chapter PDF: `08_Compatibility_and_Coupling.pdf`